00 MASTER INDEX • COMPLETE STUDY BOOK

00 MASTER INDEX — Complete C++ Roadmap & Coverage Guide

Master Navigation Index, Study Order, Uploaded Notes Audit & Standard References

Contents in This PDF

1. **README**
2. **REFERENCES**
3. **ROADMAP**
4. **TOPIC_COVERAGE**

00 — Master Index & Navigation Hub

This directory contains the master navigation, learning roadmap, classroom notes coverage tracker, and primary technical bibliography for the entire `CPP_COMPLETE_REFERENCE_LIBRARY/`.

Files in `00_MASTER_INDEX/`

1. `README.md` (`./README.md`) — Overview of the Master Index and how to use the tracking documents.
2. `ROADMAP.md` (`./ROADMAP.md`) — Semester-long learning schedule, examination preparation workflow, and topic dependency graph.
3. `TOPIC_COVERAGE.md` (`./TOPIC_COVERAGE.md`) — Detailed audit of the classroom examples, coding style conventions, identified errors/intentional compile-time errors, and module-by-module completion status.
4. `REFERENCES.md` (`./REFERENCES.md`) — Authoritative C++23 standards, cppreference documentation, C++ Core Guidelines, and compiler manual citations used throughout the library.

Master Technical References & Standards Bibliography

All explanations, language rules, evaluation-order guarantees, and library specifications in `CPP_COMPLETE_REFERENCE_LIBRARY/` are grounded in the following authoritative sources:

1. ISO C++ Standard & Working Drafts

- **Source Name**: ISO/IEC 14882:2023 (Programming Languages — C++, C++23) & Public Working Draft N4950
- **Relevant Topics**: Lexical conventions, object lifetime (`[basic.life]`), value categories (`[basic.lval]`), sequencing and evaluation order (`[intro.execution]`), class access control (`[class.access]`), derived class access (`[class.access.inh]`), operator overloading (`[over.oper]`), exception handling (`[except]`).
- **URL**: https://isocpp.org/std/the-standard (Working drafts: https://eel.is/c++draft/)
- **Information Used**: Exact rules for undefined behavior vs. ill-formed code, C++17/C++20/C++23 sequencing rules, access specifier transformations in inheritance, and exception stack unwinding guarantees.

2. cppreference.com

- **Source Name**: C++ Reference (`en.cppreference.com`)
- **Relevant Topics**: C++ Operator Precedence, Pointer Declarations, `std::unique_ptr` , `std::shared_ptr` , `std::weak_ptr` , `std::string` , `<iostream>` , `<fstream>` , `<exception>` , and Standard Template Library containers.
- **URL**: https://en.cppreference.com/w/cpp
- **Information Used**: Standard library headers, C++17/C++20/C++23 version annotations, operator precedence and associativity tables, and exception safety guarantees.

3. C++ Core Guidelines

- **Source Name**: C++ Core Guidelines (Edited by Bjarne Stroustrup and Herb Sutter)
- **Relevant Topics**: Resource management (RAII), `const` correctness, pointer vs. reference selection, class hierarchy design, and error handling (`E.*` , `R.*` , `C.*`).
- **URL**: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- **Information Used**: Best-practice comparisons showing when simple classroom constructs are sufficient and when modern C++ practices (`nullptr` , member initializer lists, `override` , `unique_ptr`) improve safety.

4. GCC & Clang Compiler Documentation

- **Source Name**: GCC Online Documentation (GNU Compiler Collection) & Clang C++ Language Compatibility Documentation
- **Relevant Topics**: Compiler diagnostics (`-Wall -Wextra -Wpedantic`), `-std=c++23` flags, access-violation error messages, and unsequenced modification warnings (`-Wsequence-point`).
- **URL**: https://gcc.gnu.org/onlinedocs/ and https://clang.llvm.org/docs/

- **Information Used**: Explaining compiler error messages on Category B (`_compile_error.cpp`) programs and warning flags for student lab verification.

B.Tech Mathematics & Computing — C++ Study Roadmap

1. Learning Progression & Prerequisites

```text
[01 Fundamentals] ──> [02 Data Types] ──> [03 Operators & Sequencing] ──> [04 Control Flow]
                                                                                  │
      ┌───────────────────────────────────────────────────────────────────────────┘
      ▼
[05 Functions] ──> [06 Pointers & References] ──> [07 Arrays & Strings] ──> [08 Dynamic Memory]
                              │
                              ▼
               [09 Classes & Objects]
                  ├──> [13 Constructors & Destructors]
                  ├──> [14 Friend Functions & Classes]
                  ├──> [12 Operator Overloading]
                  └──> [10 Inheritance] ──> [11 Polymorphism] ──> [15 Virtual Functions &
Abstraction]
                                                                                  │
      ┌───────────────────────────────────────────────────────────────────────────┘
      ▼
[16 Exception Handling] ──> [17 File Handling] ──> [18 Templates] ──> [19 Namespaces] ──> [20
Casting] ──> [21 STL & C++23]
```

2. How to Study Each Module

For every topic directory (`01` through `21`):

1. **Read `01_THEORY/` in Numerical Order**:
   - Start with `01_topic_introduction.md` for intuition, real-world analogy, and why the feature exists.
   - Study `02_core_concepts.md` and `03_syntax_and_rules.md` alongside the SVG diagrams in `05_DIAGRAMS/`.
2. **Trace `02_PROGRAMS/` Before Running**:
   - Read the program from `int main()` line by line.
   - Trace variable values on paper using the dry-run method taught in the theory files.
   - Compare Category B (`_compile_error.cpp`) files against their corrected Category A counterparts to understand *why* the compiler rejects specific code.
3. **Solve `03_PRACTICE_QUESTIONS/` Without Looking at Solutions**:
   - Attempt the **Conceptual**, **Predict the Output**, **Debugging**, and **Programming** questions on paper (simulating university written exams).
4. **Verify Against `04_DETAILED_SOLUTIONS/`**:
   - Check not just the final answer, but the *exact C++ standard rule* (e.g., access conversion rules, operator precedence vs. sequencing, stack unwinding order).

3. University Exam & Viva Preparation Strategy

- **Mid-Semester Exam Focus**: Modules `01` through `10` and `13` (Pointers, Arrays, Classes, Constructors/Destructors, and Inheritance access specifiers).
- **End-Semester Exam Focus**: Modules `10` through `21` (Inheritance ambiguity/virtual base classes, Operator Overloading, Runtime Polymorphism & vtables, Exception Handling & Stack Unwinding, File Handling, Templates, and STL).
- **Practical Lab & Viva Focus**: Modules `22_PROGRAMMING_PRACTICE`, `23_LAB_PROGRAMS`, `24_THEORY_AND_VIVA_QUESTIONS`, and `25_REVISION_NOTES`.

Classroom Notes Inspection & Complete Topic Coverage Tracker

1. Inspection of Provided Classroom Examples & Notes

Audit Note on Uploaded Classroom Material

All classroom examples provided in the specification have been inspected and analyzed below. No additional external attachment files were present in the workspace filesystem; if you have additional handwritten or PDF pages not included in the initial prompt, their specific page numbers cannot be guessed and can be appended directly to the relevant modules. Every classroom code pattern, identifier convention, and intentional error provided in your instructions is preserved and documented below.

Classroom Example 1: `person` Class (Baseline Classroom Style)

CPP

```cpp
#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    void input(int a) {
        age = a;
    }

    void display() {
        cout << "Age: " << age << endl;
    }
};

int main() {
    person p;

    p.input(20);
    p.display();

    return 0;
}
```

- **Concept Demonstrated**: Basic class definition, `private` data encapsulation (`age`), `public` member functions (`input` , `display`), stack object creation (`person p;`), and member access via the dot operator (`.`).
- **Technical Audit**:
  - Uses standard `<iostream>` , `int main()` , and `return 0;` .
  - Note for Modern C++: If `p.display()` were called *before* `p.input(20)` , `age` would have an indeterminate value (reading an uninitialized non-static data member is Undefined Behavior). In

`09_CLASSES_AND_OBJECTS/`, we preserve this exact program first and then show how default member initializers (`int age = 0;`) or constructors prevent uninitialized reads.

- **Location in Repository**: `09_CLASSES_AND_OBJECTS/02_PROGRAMS/01_classroom_person_basic.cpp`

Classroom Example 2: `super` and `sub` (`protected` Inheritance Access Error)

CPP

```cpp
#include <iostream>
using namespace std;

class super {
public:
    void show() {
        cout << "hello from super" << endl;
    }
};

class sub : protected super {
};

int main() {
    sub obj;
    obj.show(); // Intentional compilation error
    return 0;
}
```

- **Concept Demonstrated**: Access transformation under `protected` inheritance (`class sub : protected super`).
- **Technical Audit**:
  - `super::show()` is `public` in `super`.
  - Because `sub` inherits from `super` using the `protected` access specifier, all `public` members of `super` become `protected` members of `sub`.
  - Inside `main()` (which is an outside non-member function, neither `sub` nor a class derived from `sub`), calling `obj.show()` violates C++ access control rules (`[class.access.inh]`).
- **Preservation & Correction Strategy**:
  - Preserved as Category B (`02_classroom_super_sub_protected_compile_error.cpp`) with exact compiler diagnostic explanation.
  - Paired with Category A corrected versions (`03_classroom_super_sub_fixed_public.cpp` and `04_classroom_super_sub_fixed_wrapper.cpp`).
- **Location in Repository**: `10_INHERITANCE/02_PROGRAMS/`

Classroom Example 3: `person` and `student` (Inside Derived vs. Outside Derived Access)

CPP

```cpp
#include <iostream>
using namespace std;

class person {
public:
    int age;

    int input(int a) {
        age = a;
        return age;
    }
};

class student : protected person {
public:
    void display() {
        cout << input(30) << endl;
    }
};

int main() {
    student s;

    s.input(40); // Intentional compilation error

    s.display();
    return 0;
}
```

- **Concept Demonstrated**: Crucial distinction between **accessibility from inside a derived class member function** (`student::display()` calling `input(30)` — **ALLOWED** because `input` is `protected` in `student`) versus **accessibility from outside via a derived object in `main()`** (`s.input(40)` — **FORBIDDEN / COMPILE ERROR** because `protected` members are inaccessible to outside code).
- **Location in Repository**:
  - Intentional compile-error version: `10_INHERITANCE/02_PROGRAMS/05_classroom_person_student_compile_error.cpp`
  - Corrected runnable version: `10_INHERITANCE/02_PROGRAMS/06_classroom_person_student_corrected.cpp`

Classroom Example 4: `complex_system` (Binary Operator Overloading `operator+` & `Show_Num()`)

- **Concept Demonstrated**: Binary operator overloading using a member function `operator+()` , constructor initialization of real and imaginary parts, member function `Show_Num()` , implicit operator syntax (`c1 + c2`), and explicit member function call syntax (`c1.operator+(c2)`).
- **Technical Audit & Edge Case**:
  - Demonstrates returning a new `complex_system` value without mutating `c1` or `c2` .
  - Includes analysis of `const` correctness (`complex_system operator+(const complex_system& other) const`) and the signed integer overflow edge case (why negating `INT_MIN` via `-imag` when formatting

negative imaginary parts is Undefined Behavior in signed two's complement arithmetic).

- **Location in Repository**: `12_OPERATOR_OVERLOADING/02_PROGRAMS/01_classroom_complex_system.cpp` and `02_complex_system_overflow_edge_case.cpp`

---

2. Module-by-Module Completion Matrix

Module ID	Folder Name	Theory (`.md`)	Programs (`.cpp`)	Questions & Solutions	Diagrams (`.svg`)	Status
00	00_MASTER_INDEX	4	N/A	N/A	N/A	Completed
01	01_CPP_FUNDAMENTALS	3	3	4 + 3	1	Completed
02	02_DATA_TYPES_AND_VARIABLES	3	3	4 + 3	1	Completed
03	03_OPERATORS_AND_EXPRESSIONS	3	3	4 + 3	1	Completed
04	04_CONTROL_FLOW	3	3	4 + 3	1	Completed
05	05_FUNCTIONS	3	3	4 + 3	1	Completed
06	06_POINTERS_AND_REFERENCES	5	8	4 + 3 (32 Graded Qs)	4	Completed
07	07_ARRAYS_AND_STRINGS	3	3	4 + 3	1	Completed
08	08_DYNAMIC_MEMORY_ALLOCATION	3	4	4 + 3	2	Completed
09	09_CLASSES_AND_OBJECTS	3	4	4 + 3	1	Completed
10	10_INHERITANCE	4	8	4 + 3	3	Completed
11	11_POLYMORPHISM	3	3	4 + 3	1	Completed
12	12_OPERATOR_OVERLOADING	4	5	4 + 3	2	Completed
13	13_CONSTRUCTORS_AND_DESTRUCTORS	3	4	4 + 3	1	Completed
14	14_FRIEND_FUNCTIONS_AND_CLASSES	3	3	4 + 3	1	Completed
15	15_VIRTUAL_FUNCTIONS_AND_ABSTRACTION	3	4	4 + 3	2	Completed
16	16_EXCEPTION_HANDLING	4	5	4 + 3	2	Completed
17	17_FILE_HANDLING	3	3	4 + 3	1	Completed
18	18_TEMPLATES	3	3	4 + 3	1	Completed
19	19_NAMESPACES_AND_PREPROCESSOR	3	3	4 + 3	1	Completed
20	20_TYPE_CONVERSION_AND_CASTING	3	3	4 + 3	1	Completed
21	21_STL_AND_MODERN_CPP	3	3	4 + 3	1	Completed
22	22_PROGRAMMING_PRACTICE	2	4	2 + 2	1	Completed
23	23_LAB_PROGRAMS	2	6	2 + 2	1	Completed
24	24_THEORY_AND_VIVA_QUESTIONS	4	2	2 + 2	1	Completed
25	25_REVISION_NOTES	4	2	2 + 2	1	Completed